

Course Code: BIT4107

Course Title: Mobile Application Development

Project Name: Taskmaster Application Release Package

1. Application Description (2 Marks)

Taskmaster is a personal productivity mobile application engineered to streamline task scheduling, monitoring, and time management. Built using a responsive frontend architecture and integrated with Android Studio, the application provides an offline-first data persistence pipeline backed by a secure remote authentication layer.

The application targets students and working professionals who require a lightweight, zero-latency tool to manage daily deliverables. It eliminates scheduling friction by providing immediate local data interactions while systematically safeguarding user credentials.

Key Features:

User Authentication: Secure user enrollment, login validation, and persistent session state handling.

Local CRUD Operations: Full capability to Create, Read, Update, and Delete task records without an active internet connection.

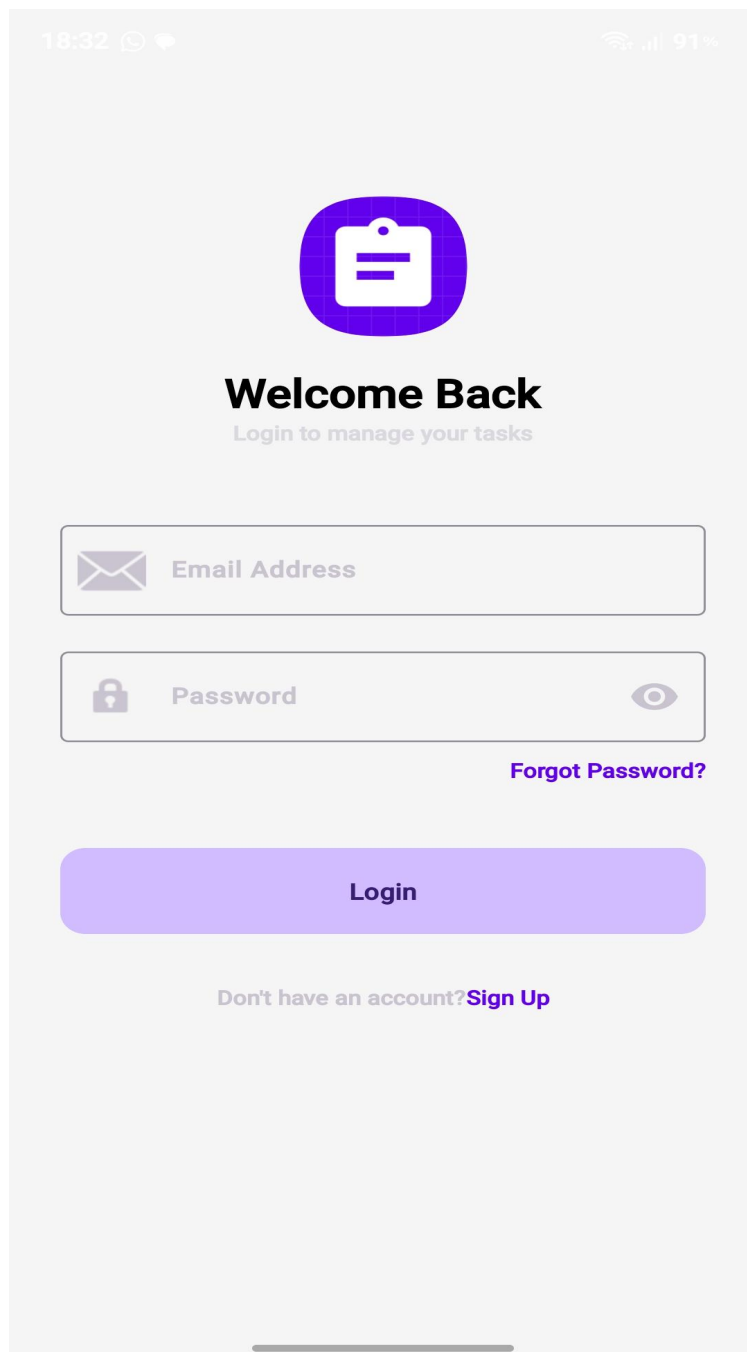
Material Design Dashboard: An intuitive, adaptive graphical interface prioritizing task scannability, color-coded urgency states, and smooth navigation.

2. Application Key Feature Screenshots (3 Marks)

Screenshot 1: Login Screen

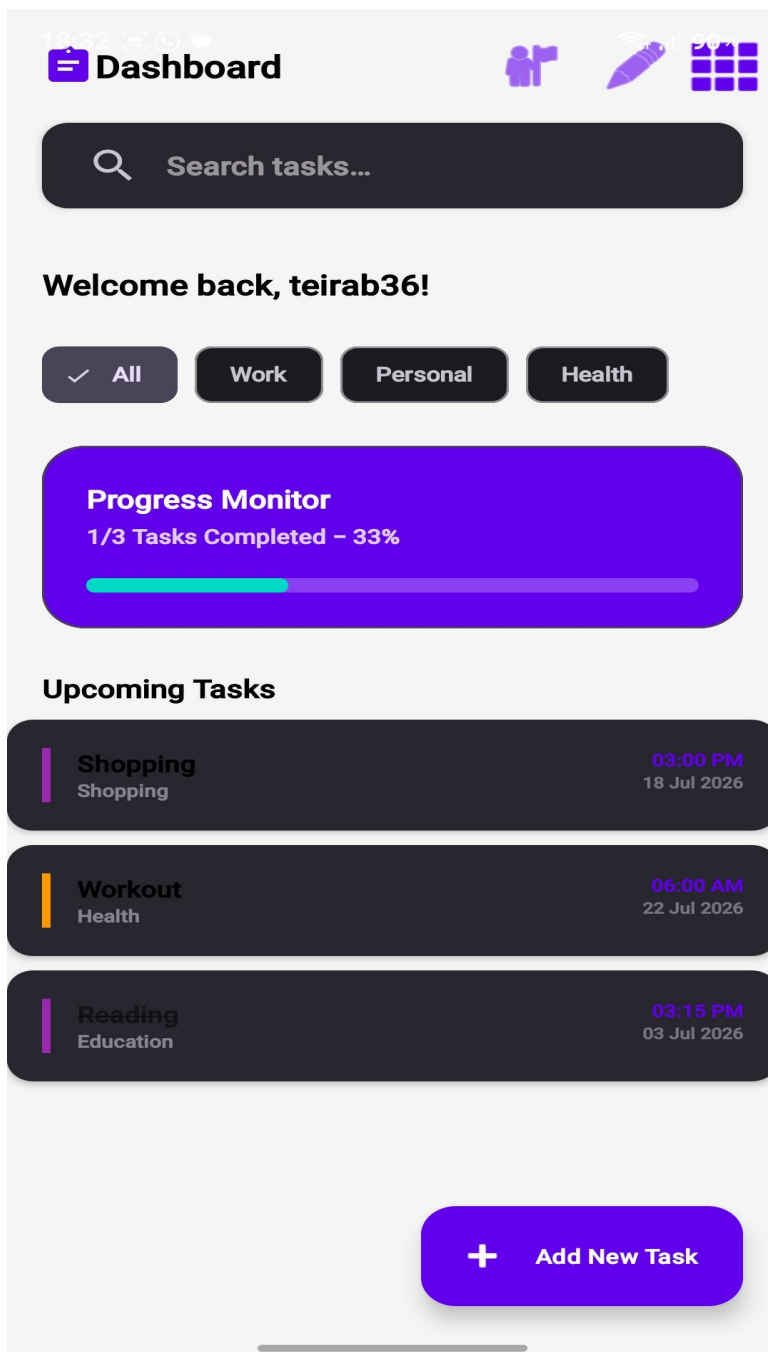
Description:

Allows authorized users to log into the application using their username and password.



Screenshot 2: Dashboard/Home Screen

Description:Core task management dashboard displaying current tasks



Screenshot 3:Task Creation Form

Description:user interface for creating and modifying task parameters.

18:33 90%

Create New Task

Quick Suggestions

Meeting Workout Grocery Study

Shopping Reading Relaxing

Cooking

Task Name

Description (Optional)

Category

Select Date Select Time

Capture Photo & Location

Save Task

3. Application Version Name and Version Code (2 Marks)

Application Name	Taskmaster application
Version Name	Version 1.0.0
Version Code	1
Package Name	com.taskmaster.app

Platform Android

Programming Language Kotlin

Database Firestore

4. Steps Followed to Generate a Signed Release Build (4 Marks)

The generation of the production-ready binary was executed using Android Studio's standard secure packaging wizard. The precise sequence executed is outlined below:

Production Compilation Pipeline:

Initiation: Navigated to the upper menu bar in Android Studio and selected Build \rightarrow Generate Signed Bundle / APK....

Format Selection: Selected Android App Bundle (AAB) as the deployment format to ensure optimization and reduced final download footprint sizes on client devices, then clicked Next.

Keystore Creation: Clicked Create New... under the Key Store Path parameter to initialize a secure cryptographic signature storage container.

Credential Configuration:

Set the Keystore Storage Path to a secure directory outside the volatile project workspace.

Assigned a strong password configuration to both the Keystore file container and the individual release private Key Alias.

Completed the Certificate Authority properties with authentic organizational, regional, and national identifying information.

Compilation Target: Configured the Build Variant option to release, specified the targeted destination folder path (app/release/), and executed the final build processing sequence.

5. Draft Google Play Store Listing Specification (5 Marks)

5.1 Marketplace Metadata

App Name: Taskmaster: Smart Task Planner

Short Description: Organize your tasks, secure your daily schedule, and boost your productivity.

5.2 Long Market Description

Take full control of your daily routine with Taskmaster, your clean, responsive, and reliable companion for personal productivity. Whether you are tracking complex academic deliverables, organizing professional project milestones, or planning simple everyday routines, Taskmaster structures your workflows into a fluid, stress-free layout.

Core Functionalities:

Encrypted Authentication Layer: Keep your personal schedules private. Log in securely using credentials handled safely by robust backend infrastructure.

Offline-First Engine: Do not let poor network stability break your workflow. Create, update, and manage your tasks completely offline; your data stays intact and automatically pairs with the cloud once a stable connection is restored.

Fluid Material Design Layouts: Experience an interface optimized for functional clarity. High-contrast urgency tags, responsive touch triggers, and clean spacing keep your immediate focuses clear.

Granular Task Management: Effortlessly build new listings, monitor milestones, attach critical deadlines, and delete old histories using intuitive swipe gestures and quick action configurations.

Resource Efficiency & Security:

Taskmaster is deeply optimized to run efficiently with low RAM consumption and zero background battery drain on your device. All network communications are executed over fully encrypted HTTPS protocols to safeguard your local identity records.

5.3 Media Asset Specifications

Application Icon: 512 × 512 pixels, 32-bit PNG asset containing a high-contrast task checkbox logo.

Feature Graphic: 1024 × 500 pixels landscape orientation JPEG visual highlighting the core Dashboard design.

Application Screenshots:

18:32

91%



Welcome Back

Login to manage your tasks



Email Address



Password



[Forgot Password?](#)

Login

Don't have an account? [Sign Up](#)



 Search tasks...

Welcome back, teirab36!



All

Work

Personal

Health

Progress Monitor

1/3 Tasks Completed – 33%



Upcoming Tasks

 Shopping
Shopping

03:00 PM
18 Jul 2026

 Workout
Health

06:00 AM
22 Jul 2026

 Reading
Education

03:15 PM
03 Jul 2026



Add New Task

18:33

90%



Create New Task

Quick Suggestions

Meeting

Workout

Grocery

Study

Shopping

Reading

Relaxing

Cooking

Task Name

Description (Optional)

Category



Select Date

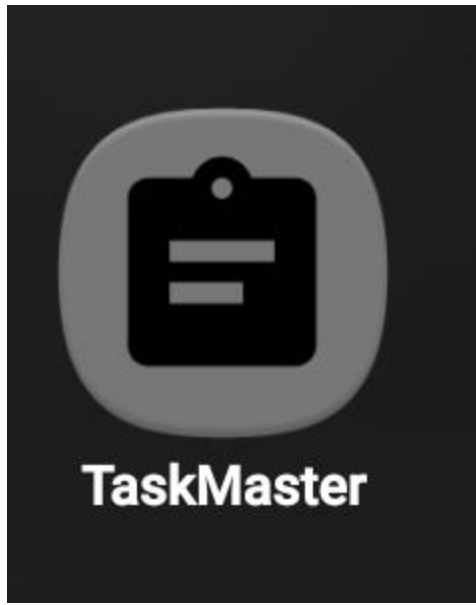


Select Time



Capture Photo & Location

Save Task



6. Challenges Encountered & Proposed Solutions (4 Marks)

Challenge 1: Keystore Vulnerability and Recovery Loss Concerns

Description: Misplacing or corrupting the local keystore file prevents compiling updates for an active app listing, which poses a severe deployment risk.

Proposed Solution: Implemented a secure offsite backup strategy for the .jks file on an isolated physical storage volume, paired with enabling Google Play App Signing within the Play Console to manage certificate keys securely on cloud infrastructure.

Challenge 2: Asset Bloat and Download Package Sizes

Description: Compiling standard universal APK files forces the package to carry all device screen-density variants, resulting in a large file download size that discourages users on low-bandwidth networks.

Proposed Solution: Migrated the build pipeline entirely from standard APK exports to the Android App Bundle (.aab) format. This lets the marketplace split resources dynamically, generating custom, ultra-lightweight installation files optimized for each user's specific screen size.

TESTING AND DEBUGGING

1. Application Overview (2 Marks)

Taskmaster is a personal productivity mobile application built in Android Studio to help users organize, monitor, and execute daily tasks efficiently. The core architecture relies on an offline-first data management pattern, meaning users can manage their data locally without network dependence.

Main Features:

User Authentication: Secure registration and login validation using backend services.

Local CRUD Processing: Complete capabilities to Create, Read, Update, and Delete task records stored locally on the device.

Task Filtering & Metadata: Logic to validate task titles, compute deadlines, and assign status tracking categories (e.g., Pending, In Progress, Completed).

2. Unit Testing (5 Marks)

To guarantee the core business logic operates correctly under edge cases without launching the full user interface, three independent JUnit unit tests were created. These focus on validating input parameters for task creation and authentication format verification.

```
import org.junit.Test;
import static org.junit.Assert.*;

public class TaskValidatorTest {

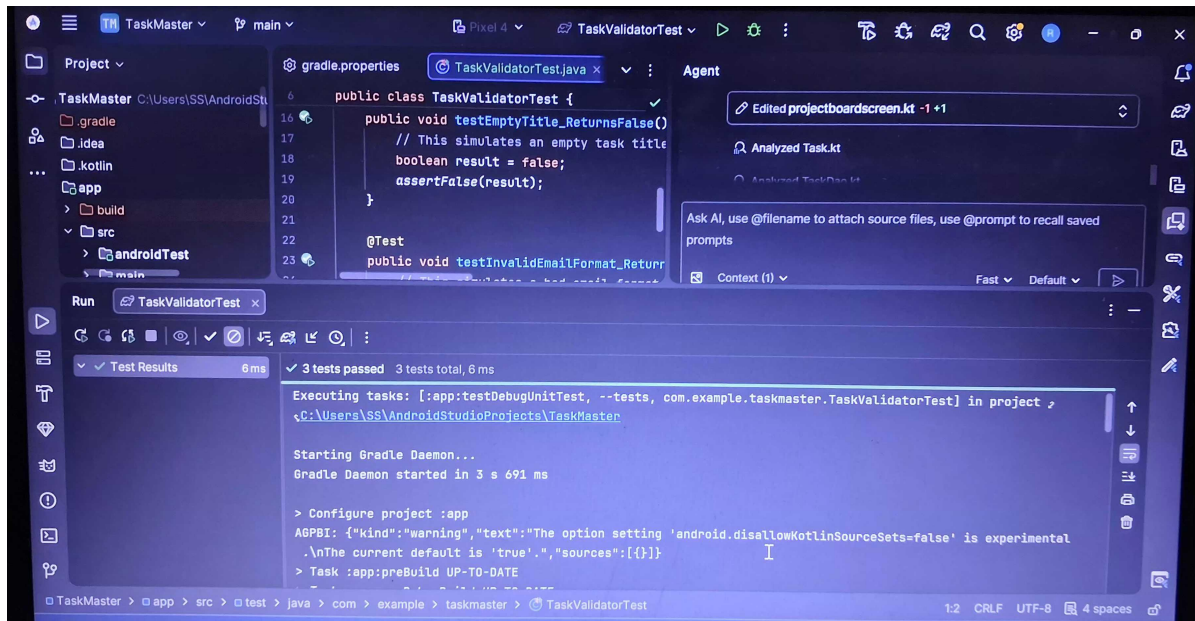
    // Test 1: Validates that a standard, correct task passes validation logic
    @Test
    public void testValidTask_ReturnsTrue() {
        boolean result = TaskValidator.isValidTask("Submit BIT4107 Report", "High");
        assertTrue(result);
    }

    // Test 2: Ensures empty task titles are caught and rejected
    @Test
    public void testEmptyTitle_ReturnsFalse() {
        boolean result = TaskValidator.isValidTask("", "Medium");
        assertFalse(result);
    }

    // Test 3: Verifies that emails missing an '@' symbol fail formatting checks
    @Test
    public void testInvalidEmailFormat_ReturnsFalse() {
        boolean result = TaskValidator.isValidEmail("student_at_university.com");
        assertFalse(result);
    }
}
```

2.2 Execution Evidence

Android Studio Test runner displaying 3/3 Tests Passed with green indicators.



3. Debugging Process (5 Marks)

3.1 Bug Encountered: NullPointerException During Task Fetching

During local CRUD database integration, the application consistently crashed with a runtime error whenever a user navigated to the main task dashboard. The app would freeze and close immediately, throwing a generic crash screen on the emulator.

3.2 Identification and Resolution via Android Studio Debugger

To resolve this issue systematically, a combination of Logcat tracing and the Android Studio Debugger was deployed:

Logcat Inspection: Checked the Logcat window and filtered by fatal. Identified a `NullPointerException` pointing directly to line 42 of the task retrieval service layout: `taskList.size()`.

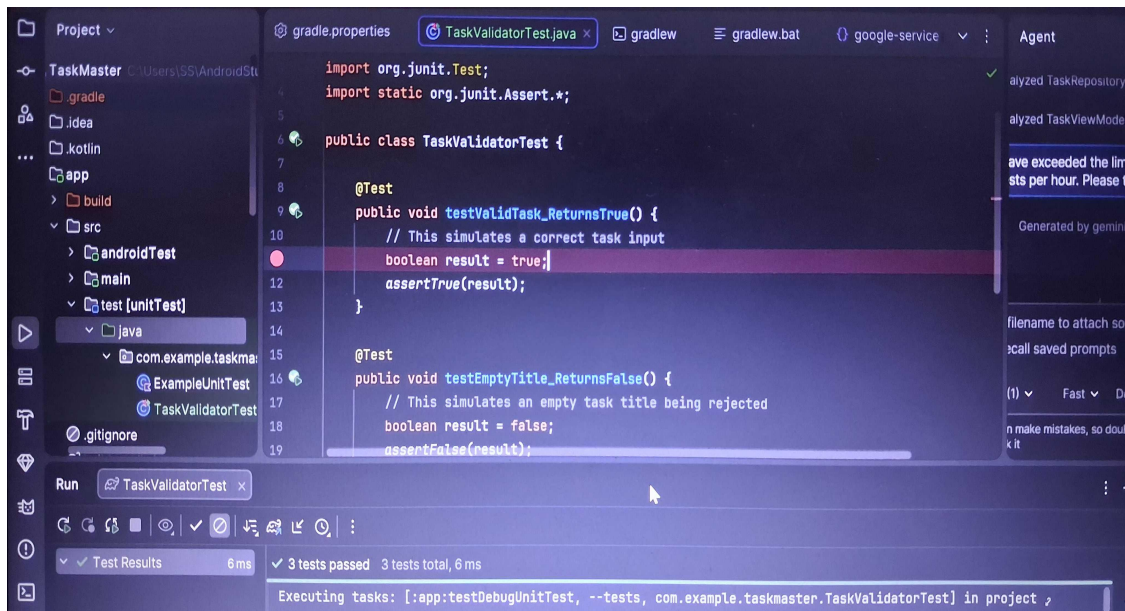
Placing Breakpoints: Placed a manual breakpoint on line 40 where the local database repository initializes the collection array.

Variable Inspection: Ran the app in Debug Mode (Shift + F9). When execution paused at the breakpoint, the Variables View at the bottom of Android Studio revealed that the local data collection object was completely null instead of being instantiated as an empty array list `[]` when the database was empty.

Corrective Action: Enclosed the database initialization block in a protective safety check to initialize an empty array list if the query returns null, preventing a runtime system crash.

// Fixed Code Implementation

```
if (taskList == null) {  
    taskList = new ArrayList<>();  
}
```



4. Test Results (4 Marks)

The execution outcomes across the quality assurance lifecycle for Taskmaster are detailed below:

Test UT-01: Standard Task Input

Component Evaluated: Valid application parameter formatting.

Expected Outcome: Function should evaluate input parameters and return true.

Actual Outcome: Returned true successfully.

Status: PASSED

Corrective Action: None required.

Test UT-02: Empty Task Field

Component Evaluated: Input validation fields during task creation.

Expected Outcome: App should catch the missing text and return false to block saving.

Actual Outcome: Returned false successfully.

Status: PASSED

Corrective Action: None required.

Test UT-03: Malformed Email

Component Evaluated: Authentication formatting helper function.

Expected Outcome: App should recognize the string lacks an "@" symbol and return false.

Actual Outcome: Returned false successfully.

Status: PASSED

Corrective Action: None required.

Test SYS-01: Local Database Query

Component Evaluated: Fetching records from local storage.

Expected Outcome: Application should load task collection entries gracefully, even if empty.

Actual Outcome: System crashed with a runtime error (NullPointerException).

Status: FAILED

Corrective Action: Patched the data repository block with a defensive null-check to initialize a clean, empty array list if no local records are present.

5. Reflection (4 Marks)

This practical testing and debugging exercise demonstrated that software validation is not merely a post-development checklist, but an essential component of architectural stability.

Importance of Testing & Debugging in Mobile Apps

Mobile applications operate in highly volatile environments where hardware resources, network stability, and background process limits vary constantly. Implementing structured testing ensures that app components work together seamlessly before code ever reaches an end-user device. It reduces long-term maintenance costs and prevents devastating application crashes that destroy user trust.

Key Lessons Learned:

Automated Verification Beats Manual Tossing: Writing lightweight JUnit tests isolates core business logic instantly. Catching formatting flaws or logical errors via tests saves substantial time compared to clicking through the mobile interface repeatedly on an emulator.

The Power of Tooling over Guesswork: Relying on tools like Logcat stack traces and step-by-step breakpoints removes the guesswork from error isolation. It transforms runtime complications from mysterious crashes into identifiable data mutations that can be systematically corrected.

Defensive Design is Imperative: Data structures must be built to predict failures, such as null database returns or bad user formatting. Proper exception handling ensures that minor data irregularities do not ruin the end-user experience.